

1. What Is OOP?

- **Definition:** OOP (Object-Oriented Programming) is a way to write code by organizing it into "objects." Objects are like real-world things (e.g., a car, a person) that have properties (like color, name) and behaviors (like driving, talking). OOP makes code easier to manage, reuse, and understand.

2. Why use OOP?

- It organizes code better.
- You can reuse code (e.g., use the same "Car" blueprint for many cars).
- It makes complex programs easier to maintain.

Example:

Let's create a simple "Dog" object.

```
# A class is like a blueprint for creating objects
class Dog:
    def __init__(self, name):
        self.name = name # Property: name

    def bark(self): # Behavior: bark
        print(f"{self.name} says Woof!")

# Creating an object (instance) of Dog
my_dog = Dog("Buddy")
my_dog.bark() # Output: Buddy says Woof!
```

3. What is a Class?

- **Definition:** A class is a blueprint for creating objects. It defines what properties (data) and behaviors (functions) the object will have.

Example:

We already saw a Dog class above. Here's another example with a Car class.

```
class Car:
    def __init__(self, brand):
        self.brand = brand # Property: brand

    def drive(self): # Behavior: drive
        print(f"The {self.brand} is driving!")

my_car = Car("Toyota")
my_car.drive() # Output: The Toyota is driving!
```

4. What is an Object?

- **Definition:** An object is something you create from a class. It's like using a blueprint (class) to build an actual thing (object).

Example:

In the above code, `my_car` is an object of the `Car` class.

5. Defining Attributes and Methods

- **Attributes:** These are the properties of an object (like name, brand).
→ **Methods:** These are the functions (behaviors) inside a class, like `bark()` or `drive()`.

Example:

Let's add more attributes and methods to a `Student` class.

```
class Student:
    def __init__(self, name, age):
        self.name = name # Attribute: name
        self.age = age   # Attribute: age

    def study(self): # Method: study
        print(f"{self.name} is studying.")

student1 = Student("Ali", 20)
print(student1.name, student1.age) # Output: Ali 20
student1.study() # Output: Ali is studying.
```

6. Instantiating Objects

- **Definition:** Instantiating means creating an object from a class. You use the class name with parentheses, like `Student("Ali", 20)`.

Example:

We've already instantiated objects like `my_dog`, `my_car`, and `student1` in the examples above.

7. The `Self` Keyword

- **Definition:** `self` refers to the object itself. It's used inside a class to access its own attributes and methods.

Example:

In the `Student` class, `self.name` means "the name of this student object."

```
class Person:
    def __init__(self, name):
        self.name = name # self refers to the object being created

    def introduce(self):
        print(f"Hi, I am {self.name}!")

person1 = Person("Sara")
person1.introduce() # Output: Hi, I am Sara!
```

8. What is a Constructor?

- **Definition:** A constructor is a special method called `__init__` that runs automatically when you create an object. It's used to set up the object's initial attributes.

Example:

The `__init__` method in all the examples above is a constructor.

9. Parameterized Constructors

- **Definition:** A constructor that takes parameters (like name, age) to initialize the object's attributes.

Example:

The `Student` class above has a parameterized constructor: `def __init__(self, name, age).`

10. Default Constructors

- **Definition:** If you don't define an `__init__` method, Python gives a default constructor that doesn't take any parameters (except `self`).

Example:

```
# A class is like a blueprint for creating objects
class Dog:
    def __init__(self, name):
        self.name = name # Property: name

    def bark(self): # Behavior: bark
        print(f"{self.name} says Woof!")

# Creating an object (instance) of Dog
my_dog = Dog("Buddy")
my_dog.bark() # Output: Buddy says Woof!
```

11. The `__del__` Method (Destructor)

- **Definition:** A destructor is a special method called `__del__` that runs when an object is deleted (when it's no longer deleted).

Example:

```
# A class is like a blueprint for creating objects
class Dog:
    def __init__(self, name):
        self.name = name # Property: name

    def bark(self): # Behavior: bark
        print(f"{self.name} says Woof!")

# Creating an object (instance) of Dog
my_dog = Dog("Buddy")
my_dog.bark() # Output: Buddy says Woof!
```

12. Class Attributes and Instance Attributes

- **Class Attributes:** Shared by all objects of a class.
- **Instance Attributes:** Unique to each object.

Example:

```
class School:
    school_name = "ABC School" # Class attribute (shared)

    def __init__(self, student_name):
        self.student_name = student_name # Instance attribute (unique)

student1 = School("Hassan")
student2 = School("Fatima")
print(student1.student_name, School.school_name) # Output: Hassan ABC School
print(student2.student_name, School.school_name) # Output: Fatima ABC School
```

13. The `__dict__` Attribute

- **Definition:** `__dict__` is a dictionary that shows all the attributes of an object and class.

Example:

```
class Person:
    def __init__(self, name):
        self.name = name

person1 = Person("Ahmed")
print(person1.__dict__) # Output: {'name': 'Ahmed'}
```

14. Methods In Python Classes

- **Instance Methods:** Work with an object (use `self`).
- **Class Methods:** Work with the class (use `@classmethod`).
- **Static Methods:** Don't need the object or class (use `@staticmethod`).

Example:

```
class MyClass:
    class_var = 10

    def instance_method(self):
        print("This is an instance method.")

    @classmethod
    def class_method(cls):
        print(f"This is a class method. Class variable: {cls.class_var}")

    @staticmethod
    def static_method():
        print("This is a static method.")

obj = MyClass()
obj.instance_method() # Output: This is an instance method.
MyClass.class_method() # Output: This is a class method. Class variable: 10
MyClass.static_method() # Output: This is a static method.
```

15. Key Principles Of OOP

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

1. Encapsulation

- It refers to **wrapping up data and methods** into a single unit (a class) and **restricting direct access** to some of an object's components.
- **Think of it like:** A capsule that contains both **medicine (data)** and **instructions (methods)** inside it — you **can't directly access** what's inside; you follow the provided interface (e.g., a method) to use it.
- **Access Modifiers:** Public (normal), Private (with `__`), Protected (with `_`).

Example:

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())  # Output: 1500
# print(account.__balance)  # Error: Can't access private attribute directly
```

→ Using `__init__` for Access Control

Definition: Use `__init__` to set up private or protected attributes.

Example:

We already did this in the `BankAccount` example above.

→ Getters And Setters Methods

Definition: Getters get the value of an attribute, and setters set the value, often with some checks.

Example:

```
class Person:
    def __init__(self, name):
        self.__name = name

    def get_name(self):  # Getter
        return self.__name

    def set_name(self, name):  # Setter
        if len(name) > 0:
            self.__name = name

person = Person("Zain")
print(person.get_name())  # Output: Zain
person.set_name("Ayesha")
print(person.get_name())  # Output: Ayesha
```

2. Inheritance

- **Definition:** Inheritance lets a class (child) take properties and methods from another class (parent).
- **Types Of Inheritance:**
- **Single Inheritance:** One parent, one child.
 - **Multi-level Inheritance:** A chain (Parent → Child → Grandchild).
 - **Multiple Inheritance:** One child with multiple parents.

Example: (Single Inheritance)

```
class Animal:
    def speak(self):
        print("I can speak!")

class Dog(Animal): # Dog inherits from Animal
    def bark(self):
        print("Woof!")

dog = Dog()
dog.speak() # Output: I can speak!
dog.bark() # Output: Woof!
```

Example: (Multi-level Inheritance)

```
class Animal:
    def speak(self):
        print("I can speak!")

class Dog(Animal):
    def bark(self):
        print("Woof!")

class Puppy(Dog): # Puppy inherits from Dog
    def play(self):
        print("I love to play!")

puppy = Puppy()
puppy.speak() # Output: I can speak!
puppy.bark() # Output: Woof!
puppy.play() # Output: I love to play!
```

Example: (Multiple Inheritance)

```
class Father:
    def skill(self):
        print("I can code.")

class Mother:
    def hobby(self):
        print("I can paint.")

class Child(Father, Mother): # Child inherits from both
    def play(self):
        print("I love playing!")

child = Child()
child.skill() # Output: I can code.
child.hobby() # Output: I can paint.
child.play() # Output: I love playing!
```

→ The **super()** Function

Definition: super() lets you call methods from the parent class.

Example:

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Call parent's __init__
        self.age = age

child = Child("Ali", 10)
print(child.name, child.age) # Output: Ali 10
```

3. Polymorphism

→ **Definition:** Polymorphism means “many forms”. It lets different classes have methods with the same name but different behaviors.

→ Types Of Polymorphism In Python

- **Method Overriding:** Child class changes a parent’s method.
- **Operator Overloading:** Changing how operators (like +) work.
- **Duck Typing:** If an object behaves like a type, it’s treated as that type.

Example: (Method overriding)

```
class Animal:
    def speak(self):
        print("I can speak!")

class Cat(Animal):
    def speak(self): # Override the parent's method
        print("Meow!")

cat = Cat()
cat.speak() # Output: Meow!
```

Example: (Operator Overloading)

```
class Point:
    def __init__(self, x):
        self.x = x

    def __add__(self, other): # Overload the + operator
        return Point(self.x + other.x)

p1 = Point(5)
p2 = Point(3)
p3 = p1 + p2
print(p3.x) # Output: 8
```

Example: (Duck Typing)

```
class Bird:
    def fly(self):
        print("Bird flies!")

class Airplane:
    def fly(self):
        print("Airplane flies!")

def make_it_fly(obj):
    obj.fly() # Works if the object has a fly() method

bird = Bird()
plane = Airplane()
make_it_fly(bird) # Output: Bird flies!
make_it_fly(plane) # Output: Airplane flies!
```

4. Abstraction

- **Definition:** Abstraction hides complex details and shows only the necessary parts. In Python, you use abstract classes to define methods that must be implemented by child classes.
- **Key Components:**
- **ABC:** Abstract Base Class (from the `abc` module) makes a class abstract.
 - **@abstractmethod** : Marks a method as abstract (must be implemented by child classes).
 - **Child Classes:** Must implement all abstract methods.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC): # Abstract class
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self): # Must implement this
        return 3.14 * self.radius * self.radius

circle = Circle(5)
print(circle.area()) # Output: 78.5
```

16. The Object Class

- In Python, **everything is an object**, and **every class** inherits from the base class `object` (either directly or indirectly).
- So, when you write:

```
class MyClass:
    pass
```

...it's actually like:

```
class MyClass(object):
    pass
```

- This means your class inherits some built-in behaviors from the object class (e.g., how it's compared, how it looks when printed, etc.).

17. Python's Unified Type System

- **Definition:** Python has a unified type system which means, **Everything in Python is an object**, and all objects are instances of classes that ultimately inherit from the base class `object`.

→ **What Does This Mean?**

In many programming languages:

- **Primitive types** (like integers, booleans, floats) are **not** treated as objects.
- But in **Python**, even these are **objects** with methods and properties.

Examples of Unified Type System in Action:

```
print(type(10))      # <class 'int'>
print(type("hello")) # <class 'str'>
print(type(True))    # <class 'bool'>
print(type([1, 2, 3])) # <class 'list'>
print(type(lambda x: x)) # <class 'function'>
print(type(object())) # <class 'object'>
```

All of these are **objects**, and inherit from the root object class.

Benefits of a Unified Type System

Benefit	Explanation
Consistency	You can treat everything the same (objects).
Polymorphism	Different objects can be used interchangeably in functions.
Introspection	You can examine any object at runtime using <code>type()</code> , <code>dir()</code> , etc.
Memory Model	Everything stored as an object reference, simplifying memory handling.

Real-World Example:

```
items = [10, "hello", [1,2], {"key": "value"}, True]

for item in items:
    print(type(item))
```

This loop works **seamlessly** because **everything is an object** and can be handled uniformly.

18. Special (Magic/Dunder) Methods

→ These are **special methods** in Python surrounded by double underscores (e.g., `__init__`, `__str__`). They allow you to define **how objects behave** with built-in operations.

Common Dunder Methods:

Dunder Method	Purpose	Example Usage
<code>__init__</code>	Constructor (called when object is created)	<code>obj = MyClass()</code>
<code>__str__</code>	String representation (used in <code>print(obj)</code>)	<code>print(obj)</code>
<code>__repr__</code>	Developer-focused string representation	<code>repr(obj)</code>
<code>__len__</code>	Length of object	<code>len(obj)</code>
<code>__getitem__</code>	Indexing like <code>obj[key]</code>	<code>obj[0]</code>
<code>__setitem__</code>	Setting value like <code>obj[key] = value</code>	<code>obj[0] = 10</code>
<code>__eq__</code>	Equality check	<code>obj1 == obj2</code>
<code>__lt__</code> , <code>__gt__</code>	Less than / Greater than	<code>obj1 < obj2</code> , <code>obj1 > obj2</code>
<code>__call__</code>	Makes object callable like a function	<code>obj()</code>
<code>__del__</code>	Destructor (called when object is deleted)	<code>del obj</code>

Example:

```
class Person:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Hello, I'm {self.name}"

    def __len__(self):
        return len(self.name)

p = Person("Sahir")
print(p)          # Hello, I'm Sahir
print(len(p))     # 5
```

Why Use Dunder Methods?

- They allow **custom classes to behave like built-in types**, making your code more Pythonic and powerful.